rsions of the simulated email that will be delivered at random times throughout the 1-week campaign.

I don't want to be included, how do I remove myself?

- All GitLab team members have the responsibility to help keep themselves, team members, customers and the company secure. As such, team members can not opt out.

Is this an invasion of privacy?

- The phishing program is covered by the Employee Privacy Policy.

Will I be publicly shamed?

- No, we will never post or create metrics which will associate a team member success or failure with a phishing simulation exercise. We will generate non-identifying metrics to be shared internally and public-facing.

I never fall for the phishing simulations received, why am I still receiving these simulation emails?

- Unfortunately, phishing emails continue to improve in deception and the ability to look genuine. By receiving phishing simulations your ability to spot the real thing is an invaluable skill to help protect yourself, team members, GitLab, and the customer.

How can I provide Feedback on my experience?

- All feedback is welcome and encouraged in this issue. The only way to continuously improve on our program is via feedback.

How to identify a basic phishing attack

When you receive an email with a link, hover your mouse over the link or view the source of the email to determine the link's true destination.

If you hover your mouse cursor over a link in Google Chrome it will show you the link destination in the status bar at the bottom left corner of your browser window.

In Safari the status bar must be enabled to view the true link destination (View -> Show Status Bar).

Some examples or methods used to trick users into entering sensitive data into phishing forms include:

- Using HTTP(S) with a hostname that begins with the name of a trusted site but ends with a malicious site.

- Using a username or password inside the request that corresponds to the name of a trusted domain and assuming the viewer won't view the whole URL.

- Using a data URI scheme instead of HTTP(S) is a particularly devious means of tricking users. Data schemes allow the embedding of an entire web page inside the URI itself. Data schemes will not show the typical green lock in the address bar of a browser that is customarily associated with a verified SSL connection.

When viewing the source of an HTML email it is important to remember that the text inside the "HREF" field is the actual link destination/target and the text before the </A> tag is the text that will be displayed to the user.

Google Login!

In this case, "Google Login!" will be displayed to the user but the actual target of the link is "evilsite.example.org".

After clicking on a link always look for the green lock icon and "secure" label that signify a validated SSL service. This icon alone is not enough to verify the authenticity of a website, however the lack of the green icon does mean you should never enter sensitive data into that website.

What to do if you suspect an email is a phishing attack

If you think an email is suspicious, it may be a phishing attempt targeted at you or GitLab, or it may be a security test. Please report the email to Security by using the PhishArm button in Gmail. It is located in the right hand sidebar of your Gmail workspace. If you don't see the button, the right hand sidebar may be collapsed and you'll need to click the arrows at the bottom right hand corner of the window to expand the sidebar.

If you are on a mobile device and using the Gmail app, the PhishArm button is toward the bottom of your Gmail app, in the available add-ons section.

If you are using another email client, you may not be able to submit the email using the PhishArm button. In this case, you may manually submit the phishing email by forwarding it to phishing@gitlab.com.

{{% note %}}
Forwarding phishing emails to phishing@gitlab.com requires additional steps by following the manual submission instructions below.
{{% /note %}}

If you use the Report Phishing button at the top right of the Gmail client, this will also require you to follow the manual submission instructions below. This is because the Report Phishing button by Gmail does not provide our security team with the email itself, and only provides us with a notification.

Submission of phishing email via PhishArm

PhishArm is found on the right side panel in Gmail. You can hide or show this panel by clicking on the > or < symbol on the bottom right corner of your web browser window

To submit an email via PhishArm to GitLab's Security Team using Gmail:

1. Select the PhishArm icon on the right-hand side toolbar in Gmail
1. Follow the instructions to report the Phish
1. Confirm you are ready to report the Phishing email
1. Receive a confirmation that the email has been forwarded to GitLab Security Team for further investigation.

Manual submission of phishing email to phishing@gitlab.com

To forward the email as an attachment to GitLab's Security Team using Gmail:

1. From a computer (not a mobile device) right click the email
1. Select Forward as attachment
1. Send it to phishing@gitlab.com

CEO & Executive Fraud

The CEO (and Executive team) will not send you an email to wire cash, or a text message to ask for gift cards, or anything else that feels like a CEO fraud or CEO scam. These types of spear attack events will be more common as we grow. Feel free to verify any unusual requests via the ceo Slack channel.

What should you do if you receive a potential phishing email or text \(smishing\) from GitLab's CEO?

1. If you are unsure whether the text or email is legitimate, contact Security to review, and confirm via the ceo Slack channel.
1. If the email is determined to be fake, follow the instructions for phishing attacks below.
1. If the text, including those received on apps like WhatsApp or Signal, is determined to be fake: block the number, notify Security, and delete the text.

- If using iOS, report the message as spam or junk

Suspicious LinkedIn profiles

Team members can verify their employment at GitLab on LinkedIn.

If a person on LinkedIn claims to work at GitLab:

1. Look for a verification badge on their LinkedIn profile. It should read GitLab: Verified using work email.
1. Look up their name on Workday, which is our Single Source of Truth for current team members. Note that alumni and those who have not yet joined GitLab will not appear in Workday.

If you believe that the profile has inaccurate information, report inaccurate information on another member's LinkedIn profile. If the user reaches out to you with a suspicious work-related message please
use the /security Slack command.

What to do if you suspect something else is suspicious

Phishing and other social engineering attacks aren't only sent via email.
You might receive a suspicious text / SMS message, a weird Direct Message on social media platforms like LinkedIn, or a phone call. If it's work related, please
use the /security Slack command.
Even if you're unsure or it feels insignificant, you can always ask in the security Slack channel.

Additional Questions, Comments, Concerns?

Please reach out to the Security Governance team!

SECTION: security/security-assurance/governance/sec-awareness-training.md

Security awareness training program

The GitLab security awareness training program provides ongoing training to GitLab team members that enhances knowledge and identification of cybersecurity threats, vulnerabilities, and attacks. Security awareness training is provided by ProofPoint, GitLab's third party provider, and will help satisfy external regulatory requirements and bolster customer assurance. The training campaigns designed to provide GitLab team members with the information they need to protect themselves and GitLab from loss or harm, highlight their role in securing GitLab on a daily basis, and empower them to make the right decisions with security best practices.

When will security awareness training occur?

New Hire security awareness training is required to be completed during onboarding. New Hire security training will be automatically assigned to you on day 1 of orientation as soon as you access the ProofPoint tile via Okta.

Annual security awareness training will occur in the second quarter of each fiscal year. Additional role-based awareness training may be assigned as needed. Prior to the security awareness training taking place, a general notification to the GitLab organization will be posted to the whats-happening-at-gitlab Slack channel.

Who will receive the security awareness training?

The successful completion of new hire and annual security awareness training is a compliance requirement for GitLab, Inc. As part of these requirements, 100% of active GitLab team members, contractors/Temporary Service Providers (TSPs), and others with access to Red, Orange and Yellow data are required to successfully complete this training.

Exceptions will be made for any individuals on extended leave at the time the campaign is launched. Upon their return from extended leave, they will be added to a catch-up campaign at a later date.

Contractors/TSPs that are able to show evidence of equivalent training completion within the calendar year will also be marked as an exception to the campaign.

For annual security awareness training, all team members hired prior to May 1 of the current year will receive an email via ProofPoint from GitLab Security <awareness@securityeducation.com> that will contain a link to access the training(s). GitLab team members hired after May 1 of the current year will have undergone New Hire security orientation training as part of their onboarding and therefore will not be required to take the annual security awareness training until the following year.

An additional Secure Coding training module must also be completed by 100% of all active GitLab team members + contractors/TSPs.

By default, GitLab team members within the Engineering Department and the sub-departments of Cost of Sales, Development, Incubation Engineering, Infrastructure and Quality that have titles with Engineer or Developer AND write code as part of their role (even Infra-as-code) will be assigned the additional training.

Other departments outside of Engineering such as Finance and Marketing also include team members that write code and will be required to complete training.

Contractors (Temporary Service Providers)

Internal Contractors/TSPs (with a GitLab email address) and external Contractors/TSPs (non-GitLab email address) that have access to production data are required to complete new hire security training during onboarding and the annual security awareness training thereafter.

Internal contractors/TSPs will be assigned training via ProofPoint. External contractors/TSPs will be sent an email with a training video and handbook links to review.

Training exceptions

- Contractors/TSPs that do NOT have access to any internal systems or sensitive data are NOT required to complete GitLab's annual training. However, they must complete training during onboarding.
- Contractors/TSPs that have been offboarded are not required to complete training, but an offboarding issue must be provided as proof of termination.
- Contractors/TSPs that are able to show evidence of equivalent training completion within the calendar year will be marked as an exception to the campaign. An exception request must

previously exist or be created.

What if a Team Member is on Extended Leave and should be marked as an Exception?

People Ops + Security Governance are collaborating to create an automated process to mark exceptions for team members that are on extended leave. However, as this is currently a manual process, the list may not be real-time. The report is provided 1 week prior to the launch of the training campaign, and the team members included on the report will be removed from the current training campaign and marked as an exception.

If a team member is not able to complete their training by the active campaign due date, an exception request is required to be submitted and acknowledged by People Ops. The team member will be marked as an exception and removed from the active campaign.

If you are a manager and are notified that one of your direct reports has not completed training, but they are on extended leave, please submit an exception request, tag @sec-governance, and provide an update. We will follow the correct procedure to remove the team member from the active campaign.

How long will the training take?

The security awareness training(s) have been limited to 30 minutes in an effort to find the best return of security investment from team members' time.

What will be covered in the training?

Security awareness training is a critical component of GitLab's security program and key to ensuring that GitLab team members are continuously educated in security core competencies.

A GitLab customized handbook first training is provided via ProofPoint. To receive full credit, the training + annual policy reviews must be completed to identify what you have learned.

What happens if training is not completed?

Team members will have up to 15 days to complete the training. If the training is not completed, Security Governance will send weekly reminder notifications requesting completion of the training.
If required, we will communicate incomplete assigned trainings to managers for assistance with completion and escalations to VPs if required. Demonstration of a completed training supports compliance with the Security awareness training program and will strengthen our regulatory requirements.

We are required to reach 100% participation for regulatory purposes. Team members that do not complete training within the required timeframe (minus exceptions) may have repercussions of their access being disabled until training has been completed. Further penalties may be incurred on a case by case basis.

Security awareness training metrics

The Security Governance team will track the annual security awareness training completion

metrics and publish them in a GitLab Issue. Once the training campaign has completed, the Security Governance team will provide results in the Security Awareness Training Program project.

Security awareness training program maturity model

GitLab leverages the SANS Security Awareness Maturity Model to gauge the maturity of its program. GitLab's current maturity level is Stage 2 - Compliance Focused.

Questions and Answers

Why are we using an external vendor?

- GitLab's internal learning platform doesn't yet have the features required to support security training requirements. The decision was made to utilize an external solution that would be able to provide robust training and a consistent process that we could rely on. We are now collaborating with ProofPoint to run our annual FY2x Security awareness training campaigns.

How will I access training?

- All users will have the ProofPoint tile added to their Okta account dashboard and managed via Single Sign-On. When training is assigned, all users will receive a personalized link to access their assigned training.

Why was I chosen?

- All GitLab team members, contractors and anyone with access to data that is NOT publicly shareable, and could expose GitLab or its customers to any harm or material impact will be required to complete our security awareness trainings whether it be during new hire orientation or annually.

I just took New Hire training, why do I have to take it again?

- We realize that recently onboarded team members have gone through the new hire security training, but there is additional content in the annual security awareness training that we think is valuable to everyone, so anyone hired prior to May 1, will be required to complete the training.

I don't want to be included, how do I remove myself?

- All GitLab team members have the responsibility to help keep themselves, team members, the company and the customer secure; all are required to complete assigned training.

Will my training status be posted publicly?

- No, we will never post or create metrics which will directly associate a team member's success or failure with a training. If we release metrics company-wide, we will generate non-identifying metrics to be shared internally and public-facing.

How can I provide Feedback on my experience?

- Please feel free to add any feedback, comments, concerns on this feedback issue.

Additional Questions, Comments, Concerns?

Please reach out to the Security Governance Team!

SECTION: security/security-assurance/governance/sec-training.md

Purpose

Security Trainings and Awareness is key to ensuring that GitLab team members are continuously provided with user education activities and exercises about evolving threats, compliance obligations, and secure workplace practices in order to refine and improve their awareness.

Scope

This standard applies to all GitLab team members, contractors/Temporary Service Providers (TSPs), consultants, vendors and other service providers that handle, manage, store or transmit GitLab data in support of GitLab's statutory, regulatory and contractual requirements.

Definitions

- GitLab Team members: users with a gitlab.com email address
- Contractors/TSPs and Consultants: Personnel who are external to GitLab who do not have a gitlab.com email address and are under a contract/agreement that involves handling, managing, storing, or transmitting GitLab data in support of GitLab's statutory, regulatory and contractual requirements.

Roles & Responsibilities

Role	Responsibilities
GitLab Team Members	Responsible for following the requirements of this standard
Security Governance Team	Responsible for the management and execution of security trainings and programs outlined in this standard
Security Governance Management	Responsible for oversight, escalation and approval of exceptions for this standard
Security Assurance Management (Code Owners)	Responsible for approving significant changes and exceptions to this standard

Standard

All GitLab Team members and contractors/TSPs are required to participate in GitLab's General Security Awareness Training, New Hire Training and on-going phishing simulations and training, or show evidence of equivalent training completion within the calendar year. Security Trainings that require participation include the following:

New Hire Security Training

New Hire Security Training is required to be completed by all GitLab Team Members and contractors/TSPs during their onboarding at GitLab. This security training provides new hires with the knowledge to identify cybersecurity threats, vulnerabilities, and attacks.

General Security Awareness Training (GSAT)

The GitLab security awareness training program provides ongoing training to GitLab team members that enhances knowledge and identification of cybersecurity threats, vulnerabilities, and attacks as well as satisfying external regulatory requirements. GitLab's handbook-first General Security Awareness Training is provided annually via Right Hand Cybersecurity, GitLab's third-party provider, and requires participation and completion by all GitLab Team Members and contractors/TSPs.

Exceptions during the active campaign will be made for GitLab team members on extended leave.

Phishing Training

The GitLab Phishing Training Program is designed to educate and evaluate GitLab's ability to detect and prevent phishing attempts. Ongoing phishing simulations and trainings are conducted once per quarter via Right Hand Cybersecurity, GitLab's third-party provider, and requires participation and completion by all assigned GitLab Team Members and contractors/TSPs.

Remember: If you see something, say something, and always report suspicious emails via PhishArm.

Data Classification Training

To maintain our culture of security and transparency, and to minimize the risk to our sensitive data and our customers, GitLab team members are encouraged to complete Data Classification Training to help understand the different types of data at GitLab and how to keep it SAFE. This is a recommended training.

Secure Coding Training

The GitLab Secure Coding Training is a required training completed by a sub-group of GitLab Team Members and contractors/TSPs in the Engineering Department. This training contains descriptions and Secure Coding Guidelines from OWASP (Open Web Application Security Project) addressing security vulnerabilities commonly identified in the GitLab codebase. This training is intended to help developers identify potential security vulnerabilities early, with the goal of reducing the number of vulnerabilities released over time.

Exceptions during the active campaign will be made for GitLab team members on extended leave.

Other Security Trainings

As our Security Training Program matures, additional trainings will be identified and added.

Exceptions

Exceptions to this procedure will be tracked as per the Information Security Policy Exception Management Process.

References

- Security Awareness Training Program
- Phishing Program
- Data Classification Standard

SECTION: security/security-assurance/governance/security-assurance-automation.md

A team supporting Security Assurance's growth

The Security Assurance program is expanding constantly, be it in breadth, headcount, complexity and scope. The Assurance department is composed of numerous teams with different processes, data sets and objectives. Having a single team focused on creating efficiencies through software allows the Department to scale faster and support the growth and impact of individual programs.

We provide to the Security Assurance department an ability to automate processes through developing scripts, building solutions and implementing fixes. Dedicated resources ensures custom solutions can be built and maintained in the future to provide continuous value-add.

How does Security Assurance Automation team operate?

Intake process

To work with the Security Assurance Automation team, please create an issue following your team's relevant link below:

Risk	Compliance	Field Security	Governance	Any other team

Work related to the request will be performed within the dedicated issue. If an Epic is needed to track the work (maybe too big for an issue, which are often scoped to be completed within an iteration ie. 2 weeks), the issue will be promoted to an Epic, and the stakeholders will be able to track the work from there.

Labels

GitLab has native features that allow team members to track their work, report on current status and view dashboard of velocity among other things.

The current limitation is that outside of GitLab, the way data is visible on our BI tool (Tableau) is through labels. Even if GitLab has native weight and stage features, using labels facilitate reporting.

These Tableau dashboards will showcase the work completed by the team, where the requests

originated from and our velocity. More advanced metrics will also be created based on that data.

Labels also help organize our issue boards in a way that ensures all issues fit into a category so no valuable work from team members isn't tracked.

In every issue

team::SAA will be added on all issues handled by the Security Assurance Automation team.

Stages

Used to track current status of issues as part of the project lifecycle. These labels also help track velocity in conjunction with Weight as they are the main inputs in our Tableau dashboard.

All SAA::Ready work should have a weight assigned to it as part of the triage/grooming process.

Don't forget to add the SAA::Complete once the work in done. Issues will be closed at the end of the iteration by closing all SAA::Complete issues still opened.

Any Issue that was added to an iteration after it started should have the Unplanned label added to it. It will help track unplanned work and how it impacts velocity and capacity.

mermaid
graph TD;
    IssueCreated== SAA::Backlog ==>IssuePrepped;
    IssuePrepped== SAA::Ready ==>IssueIterationReady;
    IssueIterationReady== SAA::In-Progress ==>IssueWorkedOn;
    IssueStalled-. SAA::In-Progress .->IssueWorkedOn;
    IssueWorkedOn== SAA::Blocked ==>IssueStalled;
    IssueWorkedOn== SAA::Complete ==>IssueCompleted;
    IssueCompleted== Issue closed ==>IssueClosed;

    style IssueCreated color:FFFFFF, fill:36454f, stroke:AA00FF
    style IssuePrepped color:FFFFFF, fill:36454f, stroke:AA00FF
    style IssueIterationReady color:FFFFFF, fill:8fbc8f, stroke:AA00FF
    style IssueWorkedOn color:FFFFFF, fill:00a0b0, stroke:AA00FF
    style IssueStalled color:FFFFFF, fill:d00000, stroke:AA00FF
    style IssueCompleted color:FFFFFF, fill:070edf, stroke:AA00FF

Label name	Meaning
SAA::Backlog	All new issues start with this label
SAA::Ready	Once groomed and reviewed, these issues can be picked up during an iteration
SAA::In-Progress	Issues are actively worked on
SAA::Blocked	Work has started but dependencies prevent further progress
SAA::Complete	All work has been completed
Unplanned	These issues weren't planned for the iteration but were picked up

Type

We use Type labels to determine what the work is about. Categories are as distinctive as possible to allow team members to quickly understand the nature of the work being done.

These categories can be changed or expanded upon depending on work items, the goal always being of accurately capturing what the SAA team member is working on.

Label name	Meaning
SAA-Type::Planning	Project Management activities
SAA-Type::Documentation	Writing, revamping, improving documentation
SAA-Type::Maintenance	Issues related to maintenance of current automations
SAA-Type::Metrics	Work related to dashboards, reporting and metrics
SAA-Type::Testing	QA, writing tests and reviewing code
SAA-Type::API-Integration	Integration between systems
SAA-Type::Process-Automation	Automating manual team processes
SAA-Type::Control-Automation	Specific automation of control testing

Source

We use Source labels to determine where the work item came from. Any additional issues stemming from that external team will always have the same source, even if the first issue came directly from.

SAA-Source::Assurance-Automation is reserved for internal work where no direct external stakeholders/dependencies exist. This label can also be used once v1 of any work has been delivered already.

Label name	Meaning
SAA-Source::Risk	Request from Risk
SAA-Source::Governance	Request from Governance
SAA-Source::Compliance	Request from Compliance
SAA-Source::Field-Security	Request from Field Security
SAA-Source::Assurance-Automation	Internal work
SAA-Source::Ad-Hoc	Request from teams not mentioned (leadership, etc.)

What does Security Assurance Automation own?

The Security Assurance Automation team is continuously engineering new automated solutions to manual processes. Below are a few projects that the team maintains.

Feedback Bot

The Feedback Bot - A bot that enables team members to send private feedback to other team members through Slack.

Escalation Engine

The Escalation engine - An engine that allows users to take automated actions on issues based on a predetermined set of criteria. The engine runs in a scheduled CI pipeline.

Dashboarding

Tableau Dashboarding - Custom dashboards using our analytic tool that integrates with data sources across GitLab.

Insight Dashboarding - Custom issue analytic dashboards native to GitLab.

Compliance control monitoring and evidence gathering automation

Conversion of manual compliance control monitoring and evidence gathering processes to partially or fully automated processes. This will save time and reduce the opportunity for human error or oversight as our control framework expands.

User Access Review Automations - Automations around UAR evidence requests (non-direct sync) and automations around access request auto creation upon change request receipt.

GitLab Project Testing and Populations - Automations around GitLab.com for gathering evidence and performing automated tests.

Software Development Lifecycle

Planning

The planning stage occurs during 1:1s, bi-weekly sprint planning meetings, Slack conversations, epic/issue comments and other channels of communication. During this stage, we gather and record the following information in the corresponding issue or epic as appropriate:

- Who is requesting the automation project?
- What are they requesting?
- Why are they requesting this project?
  - What efficiencies will be gained?
  - How much time will be saved?
- When is this project expected to be completed by?
- How is the automation expected to function?
- What is the expected time savings? (If applicable)

As a result of the planning stage, we determine the feasibility of a particular project and attempt to draw out a rough roadmap to completion.

Analysis

During the analysis stage, we continue to gather details to support accepted projects. Projects are broken down into individual components to support an agile approach to development. Those individual components are represented as child issues under the project Epic or associated tasks/issues for smaller bodies of work.

The agreed upon scale is one weight equals one business day. This means for each iteration,

team members can't have assigned more than 10 points of work.

This allows us to track unplanned work and rolled over issues for effectively and account for opportunities to better split big chunks of work into smaller manageable issues.

This approach is flexible enough to ensure the team doesn't spend valuable engineering time gauging the relevant weight to assign an issue.

If work takes less than a full business day, weight of 1 will still be used for simplicity purposes.

Weight	Level of Effort
1	Basic - Simple, typically sub-issues that can be resolved with minimal effort and have straightforward solutions. They usually don't involve dependencies.
2	Intermediate - Issues of moderate complexity that might have a few dependencies (ARs, specialized knowledge, API connections) or require some coordination amongst team members.
3	Advanced - More complex issues that have many dependencies and require coordination across teams to complete. These issues will take more time to reach a solution.
5	Challenging - Larger issue with some complexity that require specialized knowledge or substantial problem-solving. They might involve architectural designs and decisions. These issues will typically be broken down into smaller sub-issues.
8	Complex - Larger, more complex, issues that will require architectural designs and decisions. These issues are intricate, involve complex APIs, or require extensive changes. These issues will be broken down into smaller sub-issues.

Design

During the design stage, we aim to accomplish the following:

- Produce a design for the minimum viable product (MVP) solution that will satisfy the automation project's requirements.
- Design components that are modular
- Design components that can be reused to accelerate future development projects

Development and Testing

During the this stage, code is written to satisfy the requirements of a particular project. Development is accomplished in an iterative manner through many small changes. Project stakeholders may be consulted to ensure continued alignment with project expectations as code is being written.

Security Assurance Automation Engineers run tests on their code to identify bugs, vulnerabilities, and usability conflicts.

Coding Standards

When developing software, our high level objective is to follow The Zen of Python, which is a part of core Python and can be accessed by simply importing the this module. For example, at the CLI execute: python -c "import this".

Based upon technical requirements, scope, and customer deadlines, our standards can be grouped into two categories: Scripts and Modules. From a high level, a script is a .py file which is intended to be executed directly while a module is a .py file (or set of files) which is published to a PyPi registry & imported into scripts to provide more in-depth functionality.

The general philosophy is to solve new requests via scripts, which live in the scripts repository and once enough commonality is seen across multiple scripts, the functionality can be converted to a module, in an independent repository.

Templates for each of these will be found under the SAA Project Templates subgroup.

The .gitlab-ci.yml of module repositories will be used to test and package the code then publish it to GitLab's PyPi Registry. Meanwhile, in the scripts repo, LINT and security scanning will be the focus. Finally, scheduled / periodic executions should be managed in projects created under the SAA schedules subgroup.

Below is a list of libraries we use to assist with standardization:

1) All Projects
   - GitLab REST API Connection: python-gitlab
   - Logging: loguru
   - Test Framework: pytest
   - Test Coverage: coverage
     - Test Coverage (Badge): coverage-badge
   - LINT & Code Format: ruff
   - Pre-Commit Hooks
2) Scripts
   - Dependency Management: Pipenv
   - CLI: argparse
3) Modules
   - Dependency Management: PDM
     - PDM has been selected over Poetry due to PDM's direct support of PEP 621, PEP631, and PEP 517
   - CLI: click

As Simple is better than complex., this standard definition will remain minimal.

Implementation

During this stage, code is moved from the Sec Auto Dev pipeline into the Sec Auto Live pipeline. If an automation request requires web hosting or a server, the automation will live in the Sec Auto Live private GCP instance.

Once the code is ready for final review, a team member from Security Assurance will review the code and merge the branch. The project is moved to a "Done" state when the solution is operating in an automated private pipeline.

Maintenance

Routine and break-fix maintenance of automated controls and processes is performed by Security

Assurance Automation Engineers for automation related to the sub-department. Pro-active requests for maintenance can submitted through the Security Assurance Automation project.

Maintenance tasks will be tracked via GitLab Issues similar to all other automation tasks.

Maturity Model - (Non-control automation)

Maturity Model 1: At this stage, the primary objective is to establish the foundational aspects of automation functionality. The process involves a combination of both manual and automated steps. Data acquisition typically relies on manual pulling from sources, often utilizing .csv files. Basic scripts and tools are employed to execute tasks.

Maturity Model 2: In this phase, the emphasis is on advancing workflow automation by integrating more sophisticated components. Limited manual data extraction may persist, though the focus shifts towards leveraging enhanced scripting and tools. This stage is characterized by semi-automated processes that are not yet self-service and may require manual initiation.

Maturity Model 3: In this advanced stage, the primary objective is achieving a high level of automation while reducing reliance on manual data extraction. Integration with APIs is pivotal to eliminate manual intervention. The solution becomes self-service, with processes executed seamlessly within pipelines on scheduled intervals.

Control Automation Maturity

Assessing control automation maturity in a current state and also a potential state can be helpful when considering where we'd like to move control tests and monitoring to state-wise and what the estimated benefit is from a strictly time-saving standpoint. To assess this, a general qualitative and quantitative definition can be used.

When considering current vs. potential, it's important to bear in mind that control processes and/or control-test processes may require adjustment to enable automation. For example, if a current process has high variability in how it operates with inconsistent workflows and record keeping, this may be something that we are able to account for when manually assessing the design and operation of the control. But, it's a barrier to automating testing into the later maturity stages and keeps manual oversight as a critical step in control monitoring (on top of the obvious concern that a highly variable process is likely difficult to operate well and introduces significant risk of the control not operating as intended).

Control Automation Maturity Scale

| Level | Qualitative Definition
Quantitative Definition	Scalability Potential

----------|----------|----------|
| 1 | The majority of the testing process is manual, including evidence collection.
| <10% testing workload automated | Low |
| 2 | There is minor automation in place (e.g. most evidence is collected automatically or upon a simple self-service request).

| ~30% testing workload automated | Low |
| 3 | There is moderate automation in place (e.g evidence is collected and compiled almost entirely through scheduled or self-service automation. Testing decisions are entirely human generated.
| ~60% testing workload automated | Moderate |
| 4 | The control is automated to the point of decisioning for some components of the control (e.g. evidence is collected and compiled for review automatically. Some of the testing with 100% confidence in compiled evidence is conducted automatically and remaining testing is performed by a human). | ~80% testing workload automated | High |
| 5 | The control is automated entirely up to the point of human review over minor areas (e.g. evidence is collected and compiled with testing results, very few records require manual review and compiled testing provides insight to the reviewer for their consideration).
| ~90% testing workload automated | Excellent |
| 6 | The control is automated to the furthest extent. Alerting is the primary form of notice of control effectiveness. Testing documentation is done in the form of outputted reports vs. compiled tests.
| ~95% testing workload automated | Excellent |

How to use the scale?

The assessed levels are intended to be formed by input from members of Security Assurance Automation for technical input and feasibility assessment as well as members of Security Compliance for current state understanding and knowledge of where the compliance program is prioritizing in the future.

The scale can be applied across controls but more accurate and likely is that the scale is applied across system/control combinations since many controls are designed and operate differently depending on the environment.

Assessing Current Level

When assessing control automation maturity, the first step is to assess the current state of maturity. While there may be some level of subjectivity in this, the qualitative definitions should help inform the assessed current level. Generally, this level will not change unless there are significant control changes OR work is completed advancing the current level of control automation.

Once current level is assessed - add the scoped label ControlAutomationCurrentLevel::Level-X to the epic/issue.

Assessing Potential Level

Next, the potential should be assessed. There is, inevitably, subjectivity in this assessment. This is intended to be a realistic but ambitious assessment of where we think the control's testing/monitoring could be matured to. When determining a control's potential level, adding a brief justification will help support that rating decision. Something like the "final phase" of the control automation issue that is opened should already have this information as this is the end state we're working toward but if it does not already exist, please briefly justify the rating.

Potential ratings can and should change (hopefully for the better) through technology changes, process changes etc. Not every control will have a potential rating of 6 immediately, and that's OK. To reach the higher levels of maturity (4+), process changes from a control operator and assessor standpoint may need to adapt.

Once current level is assessed - add the scoped label ControlAutomationPotentialLevel::Level-X to the epic/issue.

Note: When assessing potential, try to be realistic but ambitious. If you're rating potential as a 6 because you think that if the entire platform a process is currently operated in were to change, the process could reach a level 6 but otherwise the potential level is a level 3, that may not be an appropriate rating. On the other hand, if you think a potential level is a 6 if the team that operates the control were to moderately adapt their processes, that may be an appropriate rating.

 Contact the Team

Donovan Felton, @dfelton, Security Assurance Engineer, Automation

- Automation design, development, and implementation
- GRC application administration

SECTION: security/security-assurance/governance/security-training.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

Overview

This page is for all information regarding GitLab security trainings. Security training can take the form of training for specific groups, security awareness, and also training developed by the Security Department for the added security benefits to GitLab team members, GitLab customers, and the larger security community.

Security Assurance

The Security Assurance sub department handles security training needs that involve Field Security, Security Governance, Security Compliance and Security Risk.

For more information on Security Assurance, visit the Security Assurance page.

Security Awareness Training

GitLab team members are probably most familiar with security awareness training which is a handbook first GitLab-customized training + annual policy reviews provided via ProofPoint. GitLab requires all new hires to complete New Hire security orientation training as part of the onboarding process and annual training there after.

GitLab security awareness training has been developed by GitLab Security's Governance Program. The goal of the training is to:

1. Make all GitLab team-members aware of the GitLab Security team, and familiarize them with our efforts, team structure, and people.
1. Make all GitLab team-members aware of the importance of their role in securing GitLab on a daily basis, and to empower them to make the right decisions with security best-practices.
1. Familiarize all new GitLab team-members with security-related situations that they might encounter during their tenure with the company.
1. Help all GitLab team-members internalize and reinforce the idea that reaching out to Security is an encouraging practice.

- Special topics covered:
  - Suspected phishing
  - Acceptable Use
  - Device Lost or Stolen?!
    - Slack: the /security Slack command
    - Email (Emergencies-ONLY): panic@gitlab.com
  - Data Classification
  - No Red Data on Unapproved Locations

Training Feedback

You are strongly encouraged to engage the team behind the training and provide feedback, or ask any questions related to the content of the training. You can do that through:

1. A quarterly-reviewed GitLab feedback issue for New Hire training.
1. Annual security awareness training feedback issue.
1. Sending an email to security-training@gitlab.com.

Secure Coding Training

As a DevOps company, it makes sense that we need to focus on producing secure code, and therefore training of our developers is a high priority item. There is an entire handbook page dedicated to Secure Coding training with numerous references to both required and recommended training.

Additional Security Training

There are additional training topics that do not fit into the Security Assurance and Secure Coding training areas. These are training opportunities that fall into these general categories:

- Training for Security Department team members for specific sub departments associated with high risk roles.
- Security training for other role-based training associated with high risk roles.
- Detailed explanations of certain security-related subjects to help GitLab team members (as well as customers and the security community in general) understand and reach clarity on common security topics.

These training resources are typically grouped together by access level and intended audience. As we are handbook first, we will try to make all training available for all, although there are times we will need to convey information that needs to be restricted for internal-only access. For example, a training course about "How to create a Security Training Course for GitLab" will be geared toward the Security Department, but as it will contain no internally sensitive material, it can be fully documented in the handbook and made public - benefitting GitLab as well as the security community at large. As we will develop these courses, we will list them here.

Developing Security Training Courses for GitLab

While developing security training courses for GitLab may seem somewhat daunting, it is not. And it is not restricted to the Security Department! If you are in another department and have an idea for training that involves security, you can still come up with training courses. The biggest question you might have is "how do I do this?" We've outlined some steps for you so that you can create training courses with a security aspect to them.

Basics

There are a few basics to keep in mind. They are as follows:

- Content for training can consist of numerous items, including handbook entries (obviously), videos on GitLab Unfiltered, blog posts, runbooks, projects, and so on. By creating content in any of those forms, you can create the building blocks for a potential training course.
- Gathering those individual content building blocks and arranging them together creates a training course.
- Whenever possible, try to make training courses handbook first.
- Dealing with GitLab Learn via LevelUp. By taking the LevelUp Scavenger Hunt course you will learn a lot about the entire process and creating content.

Specific to Security

There are a few fundamental differences when creating training material vs non-security training material.

- Review the GitLab Data Classification Standard so you are aware of what is considered public vs non-public data. GREEN data can be publicly shareable, any training content that is not GREEN data should not be included in publicly-accessible training material.
- New content created from scratch needs to be reviewed by the Security Department to determine what its classification is.
- Non-public training content would not go into the handbook, but could be included in a private project or runbook.

Here are a few examples to help illustrate the point:

- Training content that talks about GitLab's Red Team and their approach to how they perform their duties is fine. Details on how a Red Team assessment was performed including examples of data recovered or specific techniques against specific GitLab assets should not be (unless perhaps it has been reviewed and "sanitized" by the Security Department).
- Security-related content that involves business partnerships, specific customers, certain

company initiatives, and other related non-public information cannot be included in publicly accessible training content. A good example would be if GitLab were working with a government agency - the agency itself may have extremely strict rules about what can be made public, and something that is considered public by GitLab's own standards may not be a part of that agency's standards.
- You can have public content that points to internal private content, similar to a handbook entry that contains a link to an internal issue or restricted document. However, you cannot quote from that private content within your public content.

Any questions about classifications, public vs non-public data, and what can be included in public training content, ask. You can ask us in the security Slack channel.

Our Vision

The Security Department vision is to be the leading example in security, innovation, and transparency. Our training should be as open as possible, and if it is viewable by the public it allows us to not only help GitLab's security, we're helping others by example. Conceivably it could also be a recruiting tool to show future GitLab team members why we're such a great place to work, allowing us to continue growing as a department and a company.

SECTION: security/security-assurance/security-compliance/_index.md

Security Compliance Team Charter

Last Updated: 2025-03-20

Mission Statement

The Security Compliance Team safeguards GitLab's position as the industry's most trusted DevSecOps platform through rigorous certification management and risks & controls monitoring. We protect our customers by turning compliance requirements into competitive advantages, using our own product to demonstrate security excellence.

Value Proposition

Security Compliance maintains GitLab's position as the most trusted DevSecOps platform by providing assurance to our customers and enabling sales through certification maintenance and expansion.

Core Competencies

1. Security certifications and attestations
   - Gap Analysis Program: feasibility analysis for certification expansion
   - External Audit coordination and execution

1. Continuous Monitoring of GitLab's Security Controls which are mapped to applicable regulatory requirements and security certifications/frameworks we have committed to.

- Policy-as-code
- Automated evidence collection and control testing
- User Access Reviews
- Risk-based control testing
- PCI Internal Control Review
- FedRAMP Continuous Monitoring

1. Observation and Remediation Management

- Identify control weaknesses and gaps (observations)
- Provide remediation recommendations and guidance
- Track remediation to completion

1. Industry and Regulatory Monitoring and Insights
   - Monitoring drafts and changes to relevant laws, executive orders, directives, regulations, policies, standards, and guidelines.
   - Collaborating on responses to relevant RFIs, RFQs, RFPs, and requests for public comment.
   - Monitoring changes to government contractual language that could impact public sector security and compliance posture.

1. Dogfooding
    - We use the GitLab product to perform our core competencies
    - We recommend GitLab feature solutions to remediate observations and reduce risk
    - We provide feedback to the product by exemplifying the compliance persona.

Operating Model

We use agile program management and project management best practices to organize our work with the goal of being as efficient as possible while continuously iterating towards our objectives. Security Compliance team members are encouraged to regularly bring up feedback on how we can improve the way we work and this is a standing topic in our weekly team meeting agenda.

Core Processes

The single source of truth for all of in-progress work is the Security Compliance team top-level epic, which has detailed status updates, along with the team epic board which we use to visualize workflow status and compare to our roadmap. All work that is directly associated with our roadmap should take place in these and issues should be opened in the Security Compliance Team Issue Tracker project. This is important for two reasons: It allows us to work efficiently by centralizing and organizing our work in a single place using a robust labeling scheme and it allows us to report on various operational metrics (performance indicators).

Much of our work related to the FedRAMP Authorization Program is unfortunately not visible to the rest of GitLab due to regulatory mandates outside of our control. In order to bring as much transparency and visibility into our work, and to continue to track basic metrics, it is critical that we continue to use our epic board and issue tracker as much as possible, even if used to track high-level tasks with links to detailed issues within the authorization boundary.

How We Work

<details><summary>How We Work</summary>

Epic hierarchy

The our team top-level epic is simply a SSOT for status updates for epic assignees / directly responsible individuals (DRIs). The immediate child epics get a seccomp-roadmap label to appear in our epic board and effectively constitute our roadmap.

1. Sub-epics group tasks required to deliver an item mentioned
1. Sub-epics represent an item from the roadmap and are delivered in a specific phase
1. Sub-epics can span multiple months, but their end date should match the 'anticipated completion date' of the roadmap phase they are added to.

The diagram below shows an example of traversing the complete hierarchy:

```
mermaid
graph TD
A(Security Compliance top-level epic) --> B(SOC 2 Type 2 attestation)
B --> C([Epic])
B --> D([SOC 2 Type 2 Gap Assessment for GitLab.com])
B --> E([Expand SOC 2 TSC scope to include Availability criteria])
C --> G([Sub-epic])
E --> H([Q1 Continuous Control Monitoring])
E --> I([Q1 CCM: GitLab.com backend])
B --> F([...])
E --> J([...])
G --> K([Issue 1])
```

Epic assignee responsibilities

Each epic has a single DRI who is ultimately responsible for delivering the project. This does not mean they are doing all of the work, rather they are ensuring the work is progressing, blockers are quickly addressed or escalated if needed, and reporting on the status each week.

The DRI needs to:

1. Work with others to move issues through the boards, for example, moving from triage to in progress to complete
1. Ensure epic and any nested child epics and issues are using the appropriate labels
1. Ensure the epic meets criteria outlined in epic structure (next section)
1. Provide status updates on the epic each week including accomplishments, what's next, overall health status, and any blockers

Epic structure

Each immediate child epic under our top-level team epic must include the following (adjust quick actions as necessary):

```
markdown
```

Background

Objective

Exit criteria
- [ ]

/label ~"FY26-Q1" ~"seccomp-function::gap assessments" ~"seccomp workflow::triage" ~"team::security compliance" ~"seccomp-roadmap"
/healthstatus ontrack
/setparent &289

<!--DO NOT EDIT BELOW THIS LINE-->

<!--STATUS NOTE START-->

<!--STATUS NOTE END-->

The bottom status note comments at the bottom are important as this is what is used to automatically post status updates to these epics and the team epic.

Epic meta data to include

1. Assignee is the DRI which should be populated whenever an epic moves to seccomp workflow::in progress
1. Start date is set to the expected start date, and updated to be the actual start date when the project begins
1. Due date is set to be the expected end date
    1. The due date is set based on the Roadmap
    1. The date that a project actually ended is taken from the date that the epic was closed
1. Health status should be kept updated (on track, needs attention, at risk)

Labels are described in the Labels section.

Roadmap

All epics and issues are set with due dates according to the official roadmap.

Process to update epic due dates / roadmap items:

1. After the end of each month Security Compliance management reviews the epic (expected) due dates and works with epic assignees / DRIs to determine any roadmap changes if an epic extends beyond the epic's planned phase.
1. Management then determines roadmap adjustments so that planned work in future phases remains realistic after shifting open work.
1. Roadmap changes are shared in the next weekly sync.

Status updates

We leverage automation to ensure team members only need to provide a status update once and management only ever has to go to one place to review it. This has historically been a big problem at GitLab with epics and issues spread across various subgroups and projects.

The status for all work relating to the Security Compliance roadmap is maintained in the description of the top-level team epic so that it is visible at a glance.

Weekly status update process

DRIs should provide weekly updates for the DRI's epics according to following process:

1. Every Thursday afternoon the epic assignee / DRI of active epics (anything that is not seccomp workflow::triage) will get @mentioned in a comment on the epic asking them to reply with a status update.
1. By 17:00 UTC / 12:00 PM ET on Fridays DRIs of active epics (or the person covering if DRI is OOO) provide an update in the status section of the description of the epic regarding status of the epic including any relevant details of child epics and issues.
   - If the DRI for a child-epic is different than the epic DRI, the epic DRI is responsible for getting updates from the child-epic DRI.
     - Format for weekly update should be a brief update (~sentence or couple bullets) for each of these three items:
       - Progress since last update - Changes deployed to production, unblocked blockers, any other progress achieved.
       - Risk and Confidence - Any new blockers identified or existing blockers that persist? Any other challenges now or in the near future? How do these blockers and/or challenges affect our confidence of completing by scheduled due date per the roadmap?
       - Mitigations - What is required to overcome challenges or blockers identified? Should this be escalated to other team members, teams, executives, or domain experts?
   - Update Workflow and Health label - After each status update, the workflow label and health status should be updated. See the SecComp Team Issue Tracker Readme for details on label structure.
1. Top-Level Epic Status Update automation periodically synthesizes updates from the DRI's status update reply comment to automatically populate their epic with the status and the top-level team epic.
1. In order to ensure efficiency we will use these same status updates across any other department, division, or OKR status updates, to include broadcasts in Slack.

Backlog refinement

Prior to the start of a new quarter, the team will spend time refining the epic backlog. This process will be led by the team Manager, who will go through the epics targeted for the upcoming quarter according to the roadmap and ensure each epic contains the following information (pulling in different stakeholders to help fill in the details as necessary):

- Background (e.g. provide context and the purpose of this work, what is it and why is it relevant?)
- Objective (SMART goal explaining the plan/solution: Specific, Measurable, Achievable, Relevant, Time-bound)
- Exit criteria (break down the work into smaller, logical chunks and highlight dependencies and predecessors)

When the above information is being added, the Epic will move from Triage to Ready status. The goal is to start each quarter with our planned roadmap items for that quarter in the Ready list.

</details>

Engagement Model

- Slack
  - Feel free to tag @sec-compliance-team to reach the entire Security Compliance team
  - The sec-assurance slack channel is the best place for questions relating to our team
- Tag us in GitLab
  - @gitlab-com/gl-security/security-assurance/security-compliance

Success Metrics

Key Metric	Why It Matters	How it's Calculated	Target Thresholds	Measurement Frequency	Reporting Mechanism	Additional Notes
Median Time to Remediate	This metric tracks our ability to remediate compliance observations compared to our SLAs.	A calculation of the time between and issues being created and closed within the observation project broken out by quarter and each risk level.	High = 6 months, Medium = 1 year	Quarterly	Tableau Dashboard	n/a
TCV / ARR of new business opportunities by certification	This metric tracks demand ($) for certifications to prioritize efforts with Product and Engineering	We are working on adding a drop-down field to Salesforce to capture customer certification requests.	TBD	TBD	TBD	This is not complete. We are working with the sales team to make this possible.
Compliance posture by NIST CSF function and category (% of controls passing)	Demonstrates the level of control effectiveness in each function/category, helping management understand which areas are strong or need improvement.	We leverage the testing consculsions for the assessments completed in the fiscal year for each NIST CSF category and function area.	90% or greater passing in each function and category	Annual	Tableau Dashboard	n/a
Number of compliance findings (Observations) that are fresh	This metrics captures our ability to stay engaged with remediation owners and activities that affect our certifications and security posture.	This looks at the last updated date of the open issues in the observation management repo.	80% of issues are fresh	Real-time	Tableau Dashabord	n/a

FY26 Strategic Initiatives

Primary Focus Areas

	Objective	Key Deliverables	Timeline
1	FedRamp ATO	- Achieve Agency ATO Achieve - FedRAMP Authorized on FedRAMP Marketplace 	Ongoing in FY26
2	Certification Expansion	- Perform Gap Assessments for ISO 42001 and ISMAP - Prepare and share audit report on our posture and readiness for the certifications - Support remediatation of identified gaps 	Assessment and report - End of Q1FY26 Remediation - Ongoing in FY26
3	Control Framework Refinement	- Streamline GitLab's control framework implementation by

expanding compliance coverage, automating control management, and enhancing documentation to support future scalability.| Ongoing in FY26|

Review and Updates

This charter will be reviewed and updated quarterly to ensure alignment with:

1. GitLab Strategy
1. Security Division Mission and Vision
1. Security's Multi-year Strategy (internal only)
1. Security Assurance Mission and Vision
1. Security Assruance Multi-year Strategy - In Development

Next scheduled review: [2025-07-31]

References

- Security Certifications
- GCF Security Control Lifecycle
- GCF Security Controls
- User Access Reviews
- Observation Methodology
- Gap Analysis Program

Return to the Security Assurance Homepage

SECTION: security/security-assurance/security-compliance/access-reviews.md

Purpose

GitLab's user access review is an important control activity required for internal and external IT audits, helping to minimize threats, and provide assurance that the right people have the right access to critical systems and infrastructure. This procedure details process steps and provides control owner guidance for access reviews.

Benefits to the organization

- Reduces security risk
- Identifies dormant and/or disabled accounts
- Ensures only required team members have access to a system
- Validates users who have changed roles have not retained access no longer relevant
- Ensures terminated team members no longer can access company systems
- Supports GitLab compliance and regulatory requirements which maintains customer trust

Scope

In-Scope Systems

Security Compliance performs Access Reviews for in-scope systems based on a subset of factors. Including:

1. Criticality: Mission Critical, Business Critical

- See the tech stack for a current listing of all GitLab Systems and Vendors and their associated critical system tier.

1. Orgin: External certification impact, Internally developed systems, Sub-processors, Integrated systems, Red Vendors

Out-of-scope Systems

Systems that fall outside of the threshold of the above in-scope system factors. As a general best pract